Mobile App Development

Lab

Dimitris Riggas

riggas@ionio.gr

Department of Informatics

Ionian University



Lab 2 – React Native Development

Core Concepts & Tool Setup



Goals:

- ✓ **Node.js & NPM**: Installation, packages, `package.json`
- ✓ **Modules**: CommonJS vs ES Modules
- ✓ **React Native & Expo**: First project, structure, running in the browser
- ✓ **JSX**: Syntax & basic rules
- ✓ **Virtual DOM**: How it works & comparison with the HTML DOM
- ✓ **Hands-on**: First Expo app + hot reload

What is Node.js?

Node.js = JavaScript Runtime Environment (outside the browser)

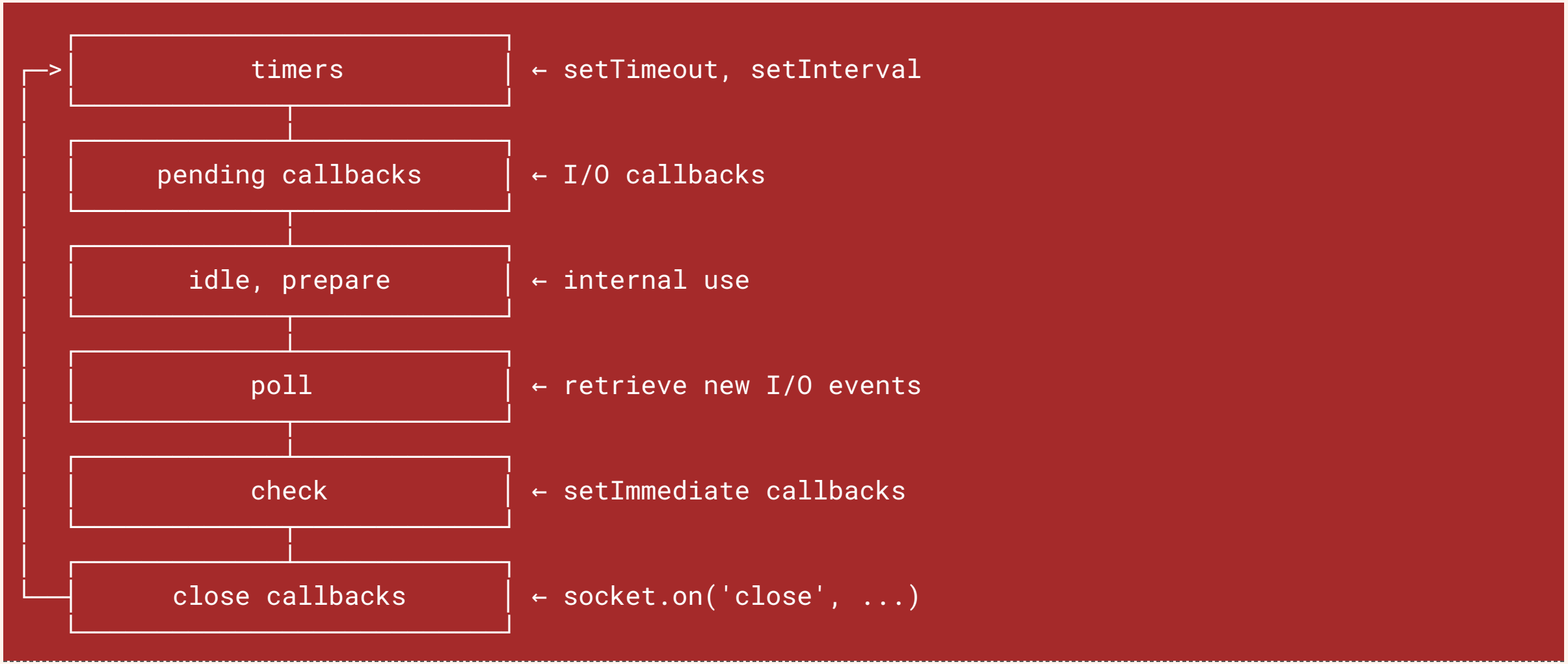
Key Features:

- **Event-driven architecture**: Built around events
- **Non-blocking I/O**: Asynchronous operations
- **V8 JavaScript Engine** (Chrome), **SpiderMonkey** (Firefox), **JavaScriptCore** (Apple)
- **Single-threaded** with an event loop for concurrency

Why Node.js for Mobile Development?

✓ One language (JavaScript) for frontend & backend
✓ Huge ecosystem (npm packages)
✓ Build tools & bundlers (Metro, Webpack)
✓ Development server for React Native

Node.js: Event Loop Architecture



“ **Source**: Node.js Official Documentation ”

Installing Node.js

Official Installer (Recommended)

- Download from nodejs.org
- Choose the **LTS version** (Long Term Support)
- **v22.13 or newer** is required (**v24 LTS** recommended) — the Expo SDK does not support older versions

Check the installation (in a **new** terminal)

```
node --version   # e.g. v24.11.0
npm --version    # e.g. 11.6.1
node             # REPL → console.log("Hi") → .exit
```

Node.js: NPM Package Manager

NPM = Node Package Manager (the largest software registry)

```
# Initialise (with default values)
npm init -y

# Install a package
npm install <package-name>
npm install chalk@4   # a specific major version

# Dev dependency
npm install --save-dev eslint

# Run a script from package.json
npm start             # or npm run <script>

# Run a tool WITHOUT a global install
npx create-expo-app@latest firstapp
```

- `npm install` → downloads into `node_modules/` and records it in `package.json`
- `^4.1.2` = compatible with 4.x.x (not 5.0.0)
- `package-lock.json` → exact versions
- `node_modules/` does **not** go into git — recreate it with `npm install`
- `npx` → runs a tool from the project, or downloads it temporarily

Node.js: Module System

CommonJS (Traditional)

```
// math.js - Module Export
module.exports = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b
};

// app.js - Module Import
const math = require('./math');
console.log(math.add(5, 3)); // 8
```

ES Modules (Modern)

```
// math.mjs - Named Export
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;

// app.mjs - Named Import
import { add, subtract } from './math.mjs';
console.log(add(5, 3)); // 8
```

- ES Modules in Node: `.mjs` files or `"type": "module"` in `package.json` · the file extension in `import` is **mandatory**
- ⚠️ Many packages are now ES Modules **only** (e.g. `chalk` ≥ 5) → `require('chalk')` does not work
- Expo / React Native uses `import` / `export` (handled by the Metro bundler)

React Native: What is it?

React Native = Framework for native mobile apps with React

Key Advantages:

✓ **Cross-platform**: iOS + Android from one codebase
✓ **Native performance**: No WebView, real native components
✓ **Hot Reloading**: See changes instantly
✓ **React ecosystem**: React libraries & patterns
✓ **JavaScript/TypeScript**: Familiar programming languages

Architecture (New Architecture, default since RN 0.76):

JavaScript Code → **JSI** (direct calls, without the old asynchronous Bridge) → Native Modules / **Fabric** renderer → Native UI

React Native CLI vs Expo

Feature	React Native CLI	Expo
Native modules	✅ Full access	✅ Through development builds / `prebuild`
Setup complexity	🔴 High (Xcode, Android Studio)	🟢 Low (browser / Expo Go)
Flexibility	🟢 Maximum	🟢 Very good
Build	🔴 Local, manual	🟢 Local or in the cloud (EAS Build)
Over-the-air updates	❌ Needs setup	✅ EAS Update

The official React Native documentation now **recommends** using a framework, i.e. **Expo**
React Native CLI: when you want full control without a framework (e.g. adding React Native to an existing native app)

Installing Expo

Expo = Managed workflow for React Native development

Setup (simple) — no global install:

```
# 1. Create a new project (plain JavaScript template)
cd ~/ReactNativeLab
npx create-expo-app@latest firstapp --template blank

# Other templates: blank-typescript, tabs, default (Expo Router + TypeScript)

# 2. Enter & start
cd firstapp
npx expo start        # or: npm start
```

⚠️ The old global `expo-cli` (`expo init`, `expo start`) has been **deprecated**.
The Expo CLI ships inside every project and is run with `npx expo ...`

Expo: Development Workflow

`npx expo start` (Metro bundler, port 8081)

```
› Metro waiting on exp://192.168.1.x:8081
› Scan the QR code above with Expo Go

› Web is waiting on http://localhost:8081

› Press a │ open Android
› Press i │ open iOS simulator
› Press w │ open web
› Press r │ reload app
› Press m │ toggle menu
› Press j │ open debugger
```

In this lab: web (`w`)

- Needs `react-dom` & `react-native-web` (`npx expo install react-dom react-native-web`)
- Save a file → **Fast Refresh**
- `console.log()` → terminal **and** browser console

In Lab 03: Expo Go on your phone

- QR code → the app on your device
- No Xcode / Android Studio needed

Expo: Project Structure

```
firstapp/
├── .git/              # already a git repository!
├── assets/            # icon, splash, favicon
├── node_modules/      # dependencies
├── App.js             # >>> Main component <<<
├── app.json           # name, version, orientation...
├── index.js           # registerRootComponent(App)
├── AGENTS.md          # instructions for AI assistants
├── package.json       # scripts & dependencies
└── package-lock.json
```

Native code (advanced)

The old `expo eject` **no longer exists**:

```
# Generate the ios/ & android/ folders
npx expo prebuild

# Development build (your own "Expo Go")
npx expo run:android
npx expo run:ios
```

- You keep all the Expo tooling
- Cloud builds: **EAS Build**

JSX: JavaScript XML

JSX = Syntax extension for JavaScript that looks like XML/HTML

Example:

```
// JSX
const element = <h1>Hello, World!</h1>;

// Transpiled to JavaScript:
const element = React.createElement(
  'h1',
  null,
  'Hello, World!'
);
```

Why JSX?

✓ **Declarative**: We describe "what" we want, not "how"
✓ **Familiar syntax**: Looks like HTML
✓ **Type safety**: Errors at compile time (with TypeScript)
✓ **Performance**: Optimisations during compilation

JSX: Basic Rules

1. Expressions in Curly Braces:

```jsx
const name = "Nikos";
const element = <h1>Hello, {name}!</h1>;

const getGreeting = (user) => `Hello, ${user}`;
const greeting = <p>{getGreeting("Maria")}</p>;
```

2. Attributes:

```jsx
// HTML-like attributes (in camelCase)
<img src={imageUrl} alt="Profile" />
<div className="container" />  // className, not class
<label htmlFor="name" />        // htmlFor, not for

// Event handlers
<button onClick={handleClick}>Click me</button>
```

JSX: Advanced Patterns

3. Conditional Rendering:

```jsx
// Ternary operator
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// Logical AND
{showWarning && <Warning message="Be careful!" />}

// If-else in a function
{renderContent()}
```

4. Lists & Keys:

```jsx
const users = ['Alice', 'Bob', 'Charlie'];

const userList = (
  <ul>
    {users.map(user => (
      <li key={user}>{user}</li>
    ))}
  </ul>
);
```

JSX in React Native

Differences from React (Web):

```jsx
// React (Web) - HTML elements
<div>
  <h1>Title</h1>
  <p>Paragraph</p>
  <button>Click</button>
</div>

// React Native - Native components
<View>
  <Text style={styles.title}>Title</Text>
  <Text>Paragraph</Text>
  <TouchableOpacity onPress={handlePress}>
    <Text>Click</Text>
  </TouchableOpacity>
</View>
```

Core Components: `View`, `Text`, `Image`, `ScrollView`, `TextInput`, `TouchableOpacity`

HTML DOM vs Virtual DOM

HTML DOM (Document Object Model):

- **Tree structure** representing the HTML document
- **Browser API** for manipulation (getElementById, querySelector, etc.)
- **Slow operations**: Reflow & repaint costs
- **Direct manipulation**: Changes go straight to the browser

DOM Manipulation Example:

```
// Vanilla JavaScript - DOM manipulation
const element = document.getElementById('myDiv');
element.style.color = 'blue';
element.innerHTML = 'New content';

// Every change → Browser reflow/repaint
```

Virtual DOM: The Concept

Virtual DOM = JavaScript representation of the real DOM

How it works:

```
1. State Change
   ↓
2. Create new Virtual DOM tree
   ↓
3. Diff algorithm (compare old vs new)
   ↓
4. Calculate minimal changes
   ↓
5. Batch update Real DOM
```

Goal: Minimise DOM work ⇒ Better performance

Virtual DOM: Diffing Algorithm

React Reconciliation Process:

```
// Initial render
<div>
  <p>Hello</p>
  <span>World</span>
</div>

// After state update
<div>
  <p>Hello</p>
  <span>Universe</span>  // Only this changed!
</div>
```

Reconciliation:

1. React compares Virtual DOM trees
2. Detects only `<span>` content changed
3. Updates ONLY that specific DOM node
4. Rest of the tree remains untouched

Virtual DOM: Performance Diagram

```
Traditional DOM Manipulation:
[State Change] → [DOM Update] → [Reflow] → [Repaint]
   (Fast)           (Slow)        (Slow)     (Slow)

   ⚠️ Each update is expensive

Virtual DOM Approach:
[State Change] → [Virtual DOM Update] → [Diffing] → [Batch DOM Update]
   (Fast)             (Fast)              (Fast)         (Slow × 1)

   ✅ Multiple changes → One DOM update
```

Key Principle:

- Virtual DOM operations: Thousands/second
- Real DOM operations: Tens/second

DOM vs Virtual DOM: Comparison

Aspect	HTML DOM	Virtual DOM
Nature	Browser API	JavaScript Object
Speed	🔴 Slow (direct manipulation)	🟢 Fast (in-memory)
Updates	Immediate (expensive)	Batched (efficient)
Reflow/Repaint	Each change	Minimized
Memory	Browser-managed	JavaScript heap
Use case	Direct browser control	React, React Native

Bottom line: Virtual DOM = Abstraction layer for performance optimisation

Preview (Lab 03): Counter App

```jsx
import React, { useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

const CounterApp = () => {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Counter: {count}</Text>

      <TouchableOpacity
        style={styles.button}
        onPress={() => setCount(count + 1)}
      >
        <Text style={styles.buttonText}>Increment</Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={styles.button}
        onPress={() => setCount(count - 1)}
      >
        <Text style={styles.buttonText}>Decrement</Text>
      </TouchableOpacity>
    </View>
  );
};
```

Preview (Lab 03): Styling

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f5f5f5',
  },
  title: {
    fontSize: 32,
    fontWeight: 'bold',
    marginBottom: 20,
    color: '#333',
  },
  button: {
    backgroundColor: '#007AFF',
    paddingHorizontal: 30,
    paddingVertical: 15,
    borderRadius: 8,
    marginVertical: 10,
  },
  buttonText: {
    color: 'white',
    fontSize: 18,
    fontWeight: '600',
  },
});
```

Debugging Tools

In the browser (Expo web):

- **F12 / Cmd+Option+I** → Browser DevTools
- 🧱 **Elements**: how `View` / `Text` become HTML elements
- 📊 **Console**: `console.log()` & errors
- 🌐 **Network**: what the browser downloads from Metro

React Native DevTools (phone / emulator):

- Press `j` in the `npx expo start` terminal
- Console, breakpoints (Sources), React Components & Profiler
- Replaces the old React Native Debugger & Flipper (both discontinued)

In the terminal:

- `console.log()` shows up there too · `r` reload · `--clear` clears the cache

And now.. *hands on*!

Continue at:

https://github.com/riggas-ionio/mobile-apps/blob/class-2026/lab02/README-EN.md



Sources

- **Node.js**: nodejs.org/docs
- **React Native**: reactnative.dev
- **Expo**: docs.expo.dev
- **React**: react.dev
- **React Native Express**: reactnative.express
- **React Native DevTools**: reactnative.dev/docs/react-native-devtools
- **Expo – Create a project**: docs.expo.dev/get-started/create-a-project